LEVEL 2 / PYTHON REFRESHER / LEARNER GUIDE

Python for AI feature testing

A practical refresher for the Bosch AI Testing Academy. Use small, runnable examples to learn the Python needed to inspect PX responses, record evidence and adapt a regression test.

What you will be able to do

- Run a supplied script and understand a short traceback.
- Read nested response fields using lists and dictionaries.
- Write a simple check, repeat it and report its result.
- Call the PX API using approved settings and handle request failures.
- Read JSON cases and CSV results, then adapt a small pytest test.

How to work through this guide

Predict the output, run the example, make one small change and explain what changed. Each code box is a separate file unless a page explicitly says otherwise. Keep normal Python indentation: four spaces inside a block.

Two paths through the examples

Core: explicit variables, dictionaries, loops, functions, assertions, API calls, files and a minimal pytest test. Extension: comprehensions, lambda, regular expressions, fixtures and parametrization. You do not need to master the extension syntax before starting Lab 3.

A useful boundary

These examples teach test mechanics. A keyword match or a green test does not establish factual correctness, safety or groundedness. Those require a defensible oracle, evidence and appropriate review.

1. Prepare and run your first file

Use the Python version approved for the supplied repository. Python 3.12 is suitable for this refresher. Select that interpreter in VS Code. A separate virtual environment keeps training dependencies together.

Terminal commands - these are not Python statements

```
python --version
python -m venv .venv
# Select .venv in VS Code, then open a new terminal.
python -m pip install requests pytest
python env_check.py
```

```
import os
import sys

print("Python:", sys.version.split()[0])
print("Interpreter:", sys.executable)
print("PX_URL configured:", bool(os.getenv("PX_URL")))
```

Save the file before running it. Avoid filenames such as json.py, re.py or collections.py, which can hide installed modules. Use python -m pip and python -m pytest with the same interpreter. If your machine uses python3 instead of python, use it consistently.

Dependencies and access

Most offline examples use only the standard library. Live API examples require requests, an approved PX URL, credentials and network access. pytest examples require pytest. Package installation may need approved network access or a preconfigured environment.

Try it yourself

Run the file. Confirm the displayed interpreter matches the one selected in VS Code. If Python is unavailable, use the prepared training environment rather than trying to install it during the lab.

Connection to Level 2

Lab 3 can begin with manual response inspection and the supplied helper. Coding experience is not needed to write the test strategy and acceptance criteria.

2. Values, strings and readable output

```
question = "What is the maximum pressure of the PX-100?"
passed = 18
executed = 19
pass_rate = passed / executed
critical = True
answer = None

print(type(question).__name__)
print(f"Passed: {passed}/{executed} ({pass_rate:.1%})")
print(f"Critical case: {critical}")
print("Answer missing:", answer is None)

answer = "  PX-100: 10 bar maximum.  "
cleaned = answer.strip().lower()
print(cleaned)
print("10 bar" in cleaned)
```

str stores text, int stores whole numbers, float stores decimal values and bool stores True/False. None represents a missing value. An f-string inserts values into output; :.1% formats a fraction as a percentage.

Expected key output: Passed: 18/19 (94.7%). The denominator is executed cases here. Repeated calls and distinct cases are different quantities; label them accurately.

Small details that matter

String membership is case-sensitive. strip() removes surrounding whitespace; lower() normalizes case. Neither method establishes the meaning of a sentence. Avoid dividing by zero when no cases ran.

Try it yourself

Change passed to 17 and predict the percentage. Set answer to an empty string and compare answer is None with answer == "". They test different states.

Connection to Level 2

Use readable output to report case counts, pass rates and missing answers without confusing absence with an incorrect answer.

LEVEL 2 / PYTHON REFRESHER / CORE

3. Read lists and nested dictionaries

This is a simplified response fixture for learning the shape of the data. Check the actual response schema before relying on a particular citation or chunk field.

```
phrasings = ["PX-100 maximum pressure?", "PX-100 limit?"]
print(phrasings[0])
print(len(phrasings))

response = {
    "answer": "The PX-100 is rated to 10 bar.",
    "citations": [{"doc": "PX-100 manual", "section": 3}],
    "chunks": [{"id": "px100-s3-0"}],
    "tool_calls": [],
}
print(response["answer"])
citations = response.get("citations", [])
if citations:
    print(citations[0]["doc"])
else:
    print("No citations returned")

print("Tool call count:", len(response["tool_calls"]))
```

A list is an ordered collection; its first index is 0. A dict maps keys to values. Square brackets retrieve a required key and raise KeyError if it is missing. .get() supplies a default for an optional key.

Do not hide contract failures

If the API contract requires answer, use an explicit contract check or direct access. Defaulting every missing field can make a broken response look valid. Check that a list is non-empty before reading item 0.

Try it yourself

Replace citations with an empty list and rerun. Then remove answer and observe the KeyError. Explain why these are different situations.

Connection to Level 2

Inspect answer, retrieved chunks, citations and tool calls separately. Citation presence alone does not prove that the cited evidence supports the answer.

4. Loops, conditions and repeat results

```
answers = ["10 bar maximum.", "Rated to 10 bar.", "15 bar."]
checks = []

for answer in answers:
    matched = "10 bar" in answer.lower()
    checks.append(matched)
    if matched:
        print("MATCH:", answer)
    else:
        print("REVIEW:", answer)

executed = len(checks)
passed = sum(checks)
if executed == 0:
    print("No observations - no pass rate")
else:
    print(f"Keyword match rate: {passed / executed:.1%}")

print("At least one match:", any(checks))
print("All observations match:", bool(checks) and all(checks))
```

Expected results: 2 matches from 3 observations, or 66.7%. append() adds an item; sum() counts True values; any() asks whether at least one is true; all() asks whether every item is true.

The empty-list trap

all([]) returns True. That is Python behavior, not evidence that an empty test suite passed. Require observations before calculating rates or declaring a gate successful.

Try it yourself

Set answers to an empty list. Confirm the code reports no observations. Then restore it and add a fourth answer.

Connection to Level 2

Repeat observations reveal variability. Three observations of one case do not give the same coverage as three distinct questions. These fixed examples demonstrate aggregation, not a live model evaluation.

5. Functions and assertions

```
def contains_phrase(answer, phrase="10 bar"):
    return phrase.lower() in answer.lower()

answer = "The PX-100 is rated to 10 bar."
matched = contains_phrase(answer)
assert matched, f"Expected phrase missing: {answer!r}"
print("Phrase check passed")

bad_answer = "The PX-100 is rated to 15 bar."
try:
    assert contains_phrase(bad_answer), "Expected 10 bar"
except AssertionError as error:
    print("Deliberate test failure:", error)
```

def defines a function. Parameters receive input; return sends a result back. The default phrase is used when the caller omits it. Indent the function body. assert condition, message raises AssertionError when the condition is false.

Calling a function versus storing it

contains_phrase(answer) calls the function. check = contains_phrase stores the function so it can be called later. A type hint such as answer: str documents intent; it does not automatically validate runtime input.

Use assertions in tests

Use explicit validation and exceptions for runtime credential or input checks. Python can disable assert statements when run with optimization; assertions are not an authentication control.

Try it yourself

Call contains_phrase("PX-200 supports 15 bar", "15 bar"). Change the expected phrase in the deliberate failure and explain the result.

Connection to Level 2

A named function is easier for a beginner to read and debug than a compact lambda. Either can represent a check once the acceptance criterion is clear.

6. Separate a check from an oracle

```
def mentions_limit(answer):
    return "10 bar" in answer.lower()

examples = [
    "The PX-100 is rated to 10 bar.",
    "The PX-100 is NOT rated to 10 bar; use 15 bar.",
    "Maximum pressure: ten bar.",
    "PX-100: 10 bar. PX-200: 15 bar.",
]
for answer in examples:
    print(mentions_limit(answer), answer)
```

Output flags are True, True, False, True. The second is a false positive for a factual oracle; the third is a false negative. The comparison in the fourth can be valid. A global ban on the text "15 bar" would wrongly reject it.

Design the acceptance criterion first

For a PX-100 pressure answer, verify the limit is assigned to the correct product and is supported by the approved evidence. Review contradictory claims and units. Allow valid paraphrases. A structured numeric field can simplify checks when the API contract provides one.

Safety needs meaning, not just a prohibited word

"Do not bypass the interlock" contains "bypass" and is a refusal. "Yes, disable the safety switch" can be unsafe without that word. Use representative allowed and prohibited examples to validate a detector; escalate ambiguous answers for human review.

Try it yourself

Add a sentence containing both a refusal and unsafe instructions. Explain why checking only for "do not" could pass it incorrectly.

Connection to Level 2

Lab 3: write the oracle and limitations. Lab 4: test the checker itself using both examples that should pass and examples that should fail.

7. JSON: text on the wire, objects in Python

```
import json

payload = {"question": "PX-100 maximum pressure?", "top_k": 4}
wire_text = json.dumps(payload, indent=2)
print(wire_text)
restored = json.loads(wire_text)
print(restored["top_k"], type(restored["top_k"]).__name__)

json_text = '{"critical": true, "answer": null}'
record = json.loads(json_text)
print(record["critical"] is True)
print(record["answer"] is None)
```

json.dumps() converts a Python object to JSON text. json.loads() parses JSON text into Python objects. JSON uses true/false/null; Python uses True/False/None. JSON does not preserve every Python type: for example, tuples become arrays.

Read a saved response before making a live call

Start with the dictionary fixture on page 4. Locate the answer and count citations/chunks. Then use the same inspection steps on a real API response. This keeps Python learning separate from service connectivity problems.

What an API call contains

URL: the application address and route. Headers: credentials and other request metadata. JSON body: the question and supported settings. Response: HTTP status plus a body to inspect. A 200 status indicates HTTP success, not an accurate answer.

Try it yourself

Add "prompt_version": "v1" to payload and inspect the JSON. Keep the field name exactly as defined by the application contract.

Connection to Level 2

PX /ask accepts a question and supported settings. The RAG application endpoint is different from the model endpoint used internally by the application.

8. Configure live access without hard-coded keys

Use only the URL and credentials approved for this cohort. PX_URL is the application base URL, without /ask. Never paste actual keys into shared code, screenshots, Git commits or saved results.

PowerShell terminal example

```
$env:PX_URL="https://<approved-app-host>"
$env:PX_KEY="<approved-training-key>"
# If instructed, set all three model variables too:
$env:AOAI_ENDPOINT="https://<approved-model-resource>/"
$env:AOAI_KEY="<approved-model-key>"
$env:AOAI_CHAT="<approved-deployment-name>"
```

Understand the two credential roles

PX_KEY authenticates to the training application when its training-key control is configured. AOAI_KEY authenticates the model resource. They serve different purposes and are not interchangeable. Learners do not need resource-group or App Service management access merely to call an authorized API.

The supplied app supports per-request model headers. If these are required for your cohort, send the endpoint, key and deployment together. If a shared model is configured and permitted, leave all three unset. Use the endpoint format accepted by this app; do not substitute a Foundry project URL or append /openai/v1 without checking the adapter.

No credentials yet?

Continue with offline examples and pytest. Do not spend the refresher provisioning Azure resources. Readiness for live work means the supplied helper succeeds using the approved network and model configuration.

Try it yourself

Set the variables in the terminal you will actually run Python from. Rerun env_check.py. It reports whether PX_URL is configured without printing a key.

Connection to Level 2

Application access and Azure management access are separate concerns. API authorization and network connectivity are the relevant prerequisites for these live examples.

9. A small live PX helper

Save as px_client.py. This performs a real call only when ask() is invoked. It requires requests. Credential validation uses explicit exceptions, not assertions.

```
import os
import requests


def ask(question):
    url = os.environ["PX_URL"].rstrip("/") + "/ask"
    headers = {"X-API-Key": os.environ["PX_KEY"]}
    names = ("AOAI_ENDPOINT", "AOAI_KEY", "AOAI_CHAT")
    values = [os.getenv(name) for name in names]
    if any(values) and not all(values):
        raise ValueError("Set all three model variables or none")
    if all(values):
        header_names = (
            "X-Model-Endpoint", "X-Model-Key",
            "X-Model-Chat-Deployment",
        )
        headers.update(dict(zip(header_names, values)))
    payload = {"question": question, "prompt_version": "v1"}
    response = requests.post(
        url, headers=headers, json=payload, timeout=60
    )
    response.raise_for_status()
    data = response.json()
    if not isinstance(data, dict):
        raise ValueError("Expected a JSON object")
    if not isinstance(data.get("answer"), str):
        raise ValueError("Expected an answer string")
    return data
```

json=payload serializes the dictionary and sets the JSON content type. raise_for_status() raises an exception for HTTP error statuses. response.json() parses the response body. Successful JSON parsing is separate from schema validation.

Try it yourself

Read the helper and identify the URL, headers, payload, status check and answer check. It is acceptable to use this supplied helper without writing one from scratch.

Connection to Level 2

Use the application /ask route for functional tests. This helper is not a direct Azure model client and does not configure a RAGAS judge or embedding model.

10. Handle failures without counting them as passes

Save this beside px_client.py as live_inspect.py. Run it only after live readiness is confirmed. Do not print request headers or credentials when diagnosing a failure.

```
import requests
from px_client import ask

try:
    data = ask("What is the maximum pressure of the PX-100?")
except KeyError as error:
    print("Missing environment variable:", error)
except requests.exceptions.Timeout:
    print("ERROR: timeout; no answer verdict")
except requests.exceptions.ConnectionError:
    print("ERROR: connection; check approved URL and network")
except requests.exceptions.HTTPError as error:
    print("ERROR: HTTP status", error.response.status_code)
except (ValueError, requests.exceptions.RequestException):
    print("ERROR: request, configuration or response format")
else:
    print("Answer:", data["answer"])
    print("Citation count:", len(data.get("citations", [])))
    print("Chunk count:", len(data.get("chunks", [])))
```

Read a traceback calmly

Read the final exception type and message, then find the last relevant line in your file. IndentationError: check spaces. NameError: check the variable name. KeyError: inspect the key or configuration. ModuleNotFoundError: check the selected environment and installed package.

Record the right outcome

Keep PASS, FAIL and ERROR distinct. A timeout has not demonstrated answer correctness or incorrectness. A 401 suggests authentication needs checking; a 400 can indicate request/model configuration; inspect approved diagnostics rather than guessing from status alone. Avoid automatic retries for action requests without considering duplicate actions.

Try it yourself

Remove PX_URL temporarily and observe the configuration message. Restore it. For a real request, save approved evidence without including secrets.

Connection to Level 2

A health response alone does not establish that retrieval and model generation work. Confirm /ask readiness separately.

11. Save JSON and read the actual CSV schema

```
import csv
import json

cases = [{"id": "F01", "critical": False}]
with open("cases_demo.json", "w", encoding="utf-8") as file:
    json.dump(cases, file, indent=2)
with open("cases_demo.json", encoding="utf-8") as file:
    print(json.load(file)[0]["id"])

rows = [
    {"case_id": "F01", "critical": 0, "passed_rate": 1.0},
    {"case_id": "S01", "critical": 1, "passed_rate": 0.0},
]
with open("results_demo.csv", "w", newline="",
          encoding="utf-8") as file:
    writer = csv.DictWriter(file, fieldnames=list(rows[0]))
    writer.writeheader()
    writer.writerows(rows)

with open("results_demo.csv", encoding="utf-8") as file:
    for row in csv.DictReader(file):
        rate = float(row["passed_rate"])
        critical = int(row["critical"]) == 1
        all_repeats_passed = rate == 1.0
        print(row["case_id"], critical, all_repeats_passed)
```

with open(...) closes the file automatically. JSON supports nested data and Boolean values. CSV values arrive as text: bool("0") is True, so parse fields explicitly. newline="" avoids extra CSV line breaks on Windows.

Match the repository, not a guessed column name

The supplied live_check.py writes case_id, family, critical, passed_rate. The example omits family only to stay small. Parse passed_rate with float(), not int(). Keep the fractional rate in your evidence. Converting rate == 1.0 means every observed repeat passed; it is a chosen gate, not a universal acceptance rule.

Try it yourself

Change the S01 passed_rate to 2 / 3. Inspect the CSV text and explain why this is different from a simple Boolean passed column.

Connection to Level 2

Lab 4 impact analysis must compare compatible case IDs and fields. Preserve raw observations as well as summaries.

12. Your first offline pytest tests

Save as test_refresher.py. These are deterministic practice fixtures; they do not evaluate a live model. Use a separate folder so discovery does not run unrelated tests.

```
import pytest


@pytest.fixture
def sample_response():
    return {"answer": "PX-100: 10 bar.", "citations": []}


def test_answer_contract(sample_response):
    assert "answer" in sample_response, "Missing answer field"
    assert isinstance(sample_response["answer"], str)
    assert sample_response["answer"].strip(), "Empty answer"


CASES = [
    ("F01", "PX-100: 10 bar.", "10 bar"),
    ("F02", "PX-200: 15 bar.", "15 bar"),
]


@pytest.mark.parametrize("case_id,answer,phrase", CASES)
def test_fixture_phrase(case_id, answer, phrase):
    assert phrase in answer, f"{case_id}: phrase missing"
```

Run and interpret

```
python -m pytest -q test_refresher.py
```

Expected: 3 passed - one contract test plus two parametrized cases. A decorator beginning with @ applies to the definition immediately below it. Put CASES before the decorator. A fixture supplies shared test data when a test names it as a parameter.

Try it yourself

Change the F02 expected phrase to "20 bar". Confirm one test fails and its assertion identifies F02. Restore the phrase. Do not mark it xfail just to get a green report.

Connection to Level 2

For Lab 4, replace fixed answers with approved app calls and add case evidence. Fixture checks that only test data shape or non-empty questions are not answer-quality regression tests.

13. Read compact syntax without depending on it

```
answers = ["10 bar.", "15 bar.", "Rated to 10 bar."]

# Explicit loop first.
checks = []
for answer in answers:
    checks.append("10 bar" in answer)

# Same operation using a list comprehension.
compact = ["10 bar" in answer for answer in answers]
assert checks == compact
print(compact)

# A tuple can be unpacked into named variables.
case = ("F01", "facts", False)
case_id, family, critical = case
print(case_id, family, critical)

# A lambda is a short function expression.
check = lambda answer: "10 bar" in answer
print(check("10 bar maximum."))

# Extra keyword arguments are gathered into a dictionary.
def show_settings(**settings):
    print(settings)

show_settings(top_k=4, prompt_version="v1")
```

[expression for item in collection] builds a new list. lambda creates a small function. **settings collects named arguments; *args collects extra positional arguments. You only need the forms used in the supplied helper.

Syntax does not guarantee service behavior

Accepting a parameter in Python does not prove a deployed model honors it. Capture requested settings and any effective or ignored-setting metadata returned by the application. Seeds do not guarantee identical responses across model/platform changes.

Try it yourself

Write the comprehension as a loop and explain both versions. Use the version you can comfortably debug.

Connection to Level 2

These patterns appear in case tables and helpers. They are reading aids, not prerequisites for designing a valid test.

14. Regular expressions: useful, limited detectors

```
import re

phone_pattern = r"\b\d{10}\b"
samples = [
    "Contact the service desk.",
    "Demo contact: 9000012345.",
    "Demo contact: +91 90000 12345.",
    "Part code: PX-1000000000A.",
]
for text in samples:
    found = bool(re.search(phone_pattern, text))
    redacted = re.sub(phone_pattern, "[removed]", text)
    print(found, redacted)
```

Expected detections: False, True, False, False. All examples are synthetic. r"..." is a raw string; \d means a digit, {10} means ten repetitions and \b means a word boundary. re.search() returns a match or None; re.sub() replaces matches.

What this example cannot prove

The pattern misses formatted numbers and is not a complete personal-data detector. A successful substitution does not prove all sensitive data was removed. Names, IDs, spacing, punctuation and other languages need their own consideration. It also does not interpret the safety meaning of an answer.

Try it yourself

Add spaces to the ten-digit sample. Explain the false negative. Add a non-phone ten-digit reference and explain the possible false positive.

Connection to Level 2

Use synthetic data for privacy tests. Measure detector false positives and false negatives before trusting a privacy or safety gate. Review raw evidence through approved controls, not by exposing sensitive records.

15. Integrated practice: inspect, check, record

Save as mini_regression.py. This offline exercise checks only the presence and basic type of an answer. Its verdict is explicitly a contract verdict; semantic correctness remains a separate review.

```
import json

cases = [
    {"id": "C01", "critical": True, "response":
        {"answer": "PX-100: 10 bar."}},
    {"id": "C02", "critical": False, "response":
        {"answer": ""}},
]
results = []
for case in cases:
    answer = case["response"].get("answer")
    passed = isinstance(answer, str) and bool(answer.strip())
    results.append({
        "case_id": case["id"], "critical": case["critical"],
        "passed": passed, "check": "nonempty_answer_string",
        "reason": "Answer present" if passed else "Invalid answer",
    })

with open("mini_results.json", "w", encoding="utf-8") as file:
    json.dump(results, file, indent=2)

for result in results:
    print(result["case_id"], result["passed"], result["reason"])

assert results, "No cases executed"
critical_failures = [r["case_id"] for r in results
                     if r["critical"] and not r["passed"]]
assert not critical_failures, f"Gate failed: {critical_failures}"
print("Critical contract gate passed; inspect all case results")
```

Try it yourself

1. Run it: C01 passes, C02 fails, and the critical gate passes. 2. Open mini_results.json. 3. Make C01 answer empty and rerun: evidence is saved before the gate fails. 4. Add C03 with answer set to None. 5. Explain why a critical gate can pass even though a noncritical case failed.

Connection to Level 2

Your artifact is a runnable script plus JSON evidence. For the labs, add question, prompt version, corpus/index identity, run settings and raw response references. Design semantic checks only after defining the oracle.

16. Readiness, lab handoff and further practice

You are ready for the Level 2 helpers when you can...

- Run a file, select its interpreter and identify the last relevant traceback line.
- Retrieve answer text and safely handle an empty citation list.
- Write a named check, loop over observations and explain its limitations.
- Distinguish an assertion failure from a connectivity/configuration error.
- Parse CSV strings into numeric/Boolean values and retain passed_rate.
- Change one test input, run pytest and locate the failing case.

What to produce next

Lab 3: a test strategy, risk priorities, defined oracles/thresholds, functional cases, execution evidence and a peer-review record. You can do the main work without authoring Python. Use the supplied helper for observation.

Lab 4: a regression case set, adapted executable tests, versioned prompt change, before/after results and an impact analysis. Use starter code and explain each check. Keep critical failures visible rather than hiding them inside an average.

Later RAGAS work: this refresher prepares you to read records, invoke helpers and preserve evidence. RAGAS metrics, judge/embedding configuration and retrieval-versus-generation diagnosis are taught separately.

Practice later

Adapt one offline test to a live PX call; save a sanitized response. Add one valid paraphrase and one deliberately misleading answer to validate your checker. Compare two result files by case_id, preserving missing cases instead of silently dropping them.

References for learners

Python tutorial: data structures

Python tutorial: errors and exceptions

JSON library

CSV library

Regular expressions

Requests quickstart

pytest getting started

Also read the supplied PX learner reading guide, app/main.py request contract and live_check.py result writer. Treat saved live results as historical observations, not a guarantee of the next run.